

CNN et limites structurelles pour les documents

Rappel convolution · Biais inductifs · Architectures · CRNN

Pourquoi revisiter les CNN ?

L'objectif : comprendre les biais inductifs — les hypothèses que l'architecture encode sur les données

Le biais inductif

Hypothèses implicites encodées dans une architecture sur la structure des données

Permet la généralisation depuis un nombre fini d'exemples

Sans biais : apprentissage impossible — théorème no free lunch (Wolpert, 1997)

Quand les hypothèses sont vraies : le modèle apprend vite et bien

Quand elles sont fausses : il échoue systématiquement

Les biais des CNN

Localité : un filtre $k \times k$ ne voit que k^2 pixels voisins

Invariance à la translation : même filtre appliqué partout

Partage des paramètres : les features sont supposées identiques en tout point

→ Très bien adaptés aux images naturelles

→ Mal adaptés aux manuscrits médiévaux

Enjeu pour ce cours

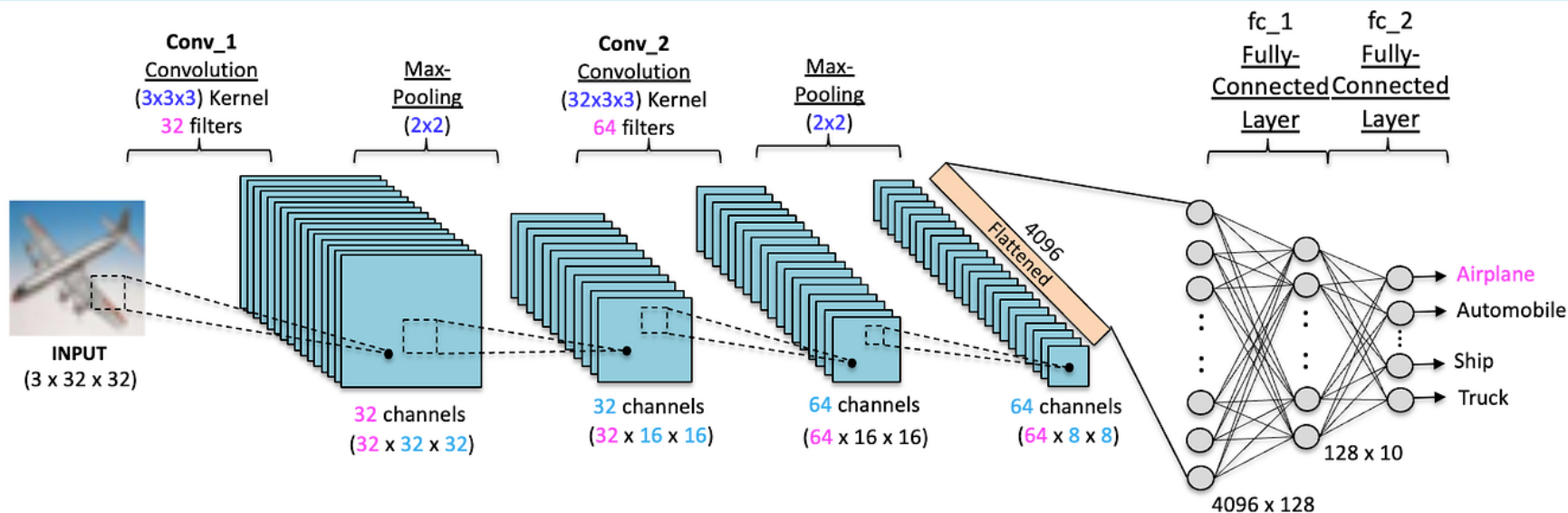
Comprendre précisément pourquoi les CNN échouent sur les manuscrits — pas juste « les Transformers font mieux »

Les CNN ne sont pas mauvais : ils font de bonnes hypothèses sur les images naturelles. Les manuscrits ne sont pas des images naturelles.

Cette analyse est la base pour comprendre pourquoi le mécanisme d'attention des Vision Transformers est plus adapté (section 2.2).

1. La convolution 2D

Formalisme, paramètres et champ réceptif



Réseau à convolution

La convolution 2D — définition et paramètres

$$F[i,j] = \sum I[i \cdot s + m, j \cdot s + n] \cdot K[m,n] + b \quad (\text{techniquement : corrélation croisée, filtre non retourné})$$

Le filtre (kernel)

Appris par rétropropagation
Filtre 3×3 : 9 paramètres +
biais

Couche avec Cout filtres / Cin
canaux :

$\text{Cout} \times \text{Cin} \times k^2 + \text{Cout}$
paramètres

Pas défini à la main —
entièrement appris

Stride s

Pas de déplacement du filtre

$s = 1$: pas de réduction de
résolution

$s = 2$: résolution divisée par 2

Alternative au pooling pour
sous-échantillonner

Padding p

valid ($p=0$) : sortie plus petite

same ($p = \lfloor k/2 \rfloor$) : résolution
préservée
(pour $s=1$ — convention
moderne)

$$H' = \lfloor (H - k + 2p)/s \rfloor + 1$$

Dilatation (« trous »)

Zéros intercalés entre éléments
du filtre

$$F[i,j] = \sum I[i + d \cdot m, j + d \cdot n] \cdot K[m,n]$$

↑ champ réceptif sans ↑
paramètres
ni ↓ résolution spatiale

Utilisé dans DeepLab, WaveNet

Le champ réceptif — théorique vs effectif

Région de l'image d'entrée qui influence un neurone donné — notion centrale pour comprendre les limites des CNN

Champ réceptif théorique

Sans pooling : $RF(l) = 1 + l \times (k-1) = 1 + 2l$ ($k=3$)

→ Croissance linéaire avec la profondeur

Avec pooling $\times 2$: $RF_{eff} \approx k^l$

→ Croissance exponentielle

Calculé algébriquement à partir de l'architecture

Champ réceptif effectif (Luo et al., 2016)

Les gradients se distribuent en gaussienne dans le champ théorique

Seul le centre contribue significativement

$RF_{effectif} \approx \sqrt{RF_{théorique}}$

Champ théorique de 121 → effectif ≈ 11

Le réseau « voit » beaucoup moins qu'on ne le croit

Implication directe pour les manuscrits médiévaux

Page à 400 DPI $\approx 4\,000 \times 5\,500$ pixels → champ réceptif nécessaire : 4 000 px

Avec blocs 3×3 + pooling $\times 2$: $\log_2(4000) \approx 12$ étapes → réduction par 4 096 → ~ 1 pixel par page

L'information spatiale fine (détail des lettres, structure locale) est irrémédiablement perdue au sommet du réseau

Pooling et normalisation

Sous-échantillonnage, invariance et stabilisation de l'entraînement

Max Pooling / Average Pooling

Réduction de résolution par sélection (max) ou moyenne

Induit une invariance locale à la translation
→ Décaler une feature dans la fenêtre ne change pas la sortie

Force pour les images naturelles (chat = chat, peu importe où)
Problème pour l'HTR : la position est porteuse de sens

Batch Normalization (Ioffe & Szegedy, 2015)

Normalise par canal sur le mini-batch : $\hat{x}_c = (x_c - \mu_c) / \sqrt{\sigma_c^2 + \epsilon}$

Puis transformation affine apprise : $y_c = \gamma_c \cdot \hat{x}_c + \beta_c$

↑ stabilité des gradients · ↑ taux d'apprentissage · régularisation implicite

Limite : se comporte mal pour petits batch sizes ($N < 8$)

Layer Normalization

Normalise sur les dimensions de features (pas sur le batch)

Indépendante de la taille du batch

→ Préférée dans les Transformers (séquences de longueur variable)

BatchNorm ↔ CNN / LayerNorm ↔ Transformer

À retenir

Le pooling crée de l'invariance → utile pour la classification, problématique pour la localisation fine et l'HTR
BatchNorm est un pilier des CNN modernes mais inadapté aux Transformers → LayerNorm prend le relais

2. Grandes architectures CNN

Innovations cumulatives qui ont façonné le domaine

LeNet, AlexNet, VGGNet — les fondations

LeNet-5 (LeCun, 1998)

1ère architecture CNN opérationnelle à grande échelle

Conçue pour MNIST (chiffres manuscrits, 28×28 px)

2 conv (5×5) + 2 pool + 3 FC · ~60 000 paramètres

Innovation : features hiérarchiques apprises par rétropropagation > features artisanales

Ironie : le 1er CNN est né pour lire des chiffres manuscrits

AlexNet (Krizhevsky, 2012)

Point d'inflexion historique : ILSVRC 2012

Erreur top-5 : 15.3% vs 26.2% (2ème) — réduction de 41%

Innovations :

→ Entraînement GPU (2× GTX 580)

→ ReLU : convergence 6× plus rapide que tanh

→ Dropout (0.5) sur les FC

→ Data augmentation

~62 M paramètres

VGGNet (Simonyan, 2014)

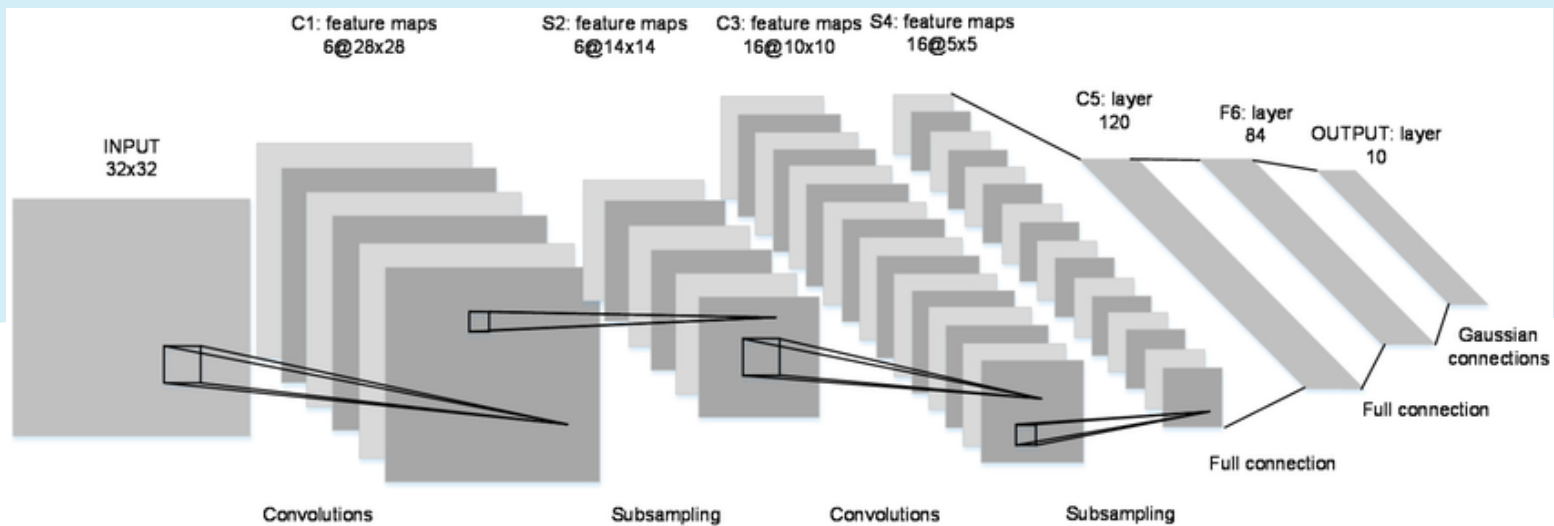
Démonstration : la profondeur est le facteur clé

Petit filtres 3×3 empilés > grands filtres

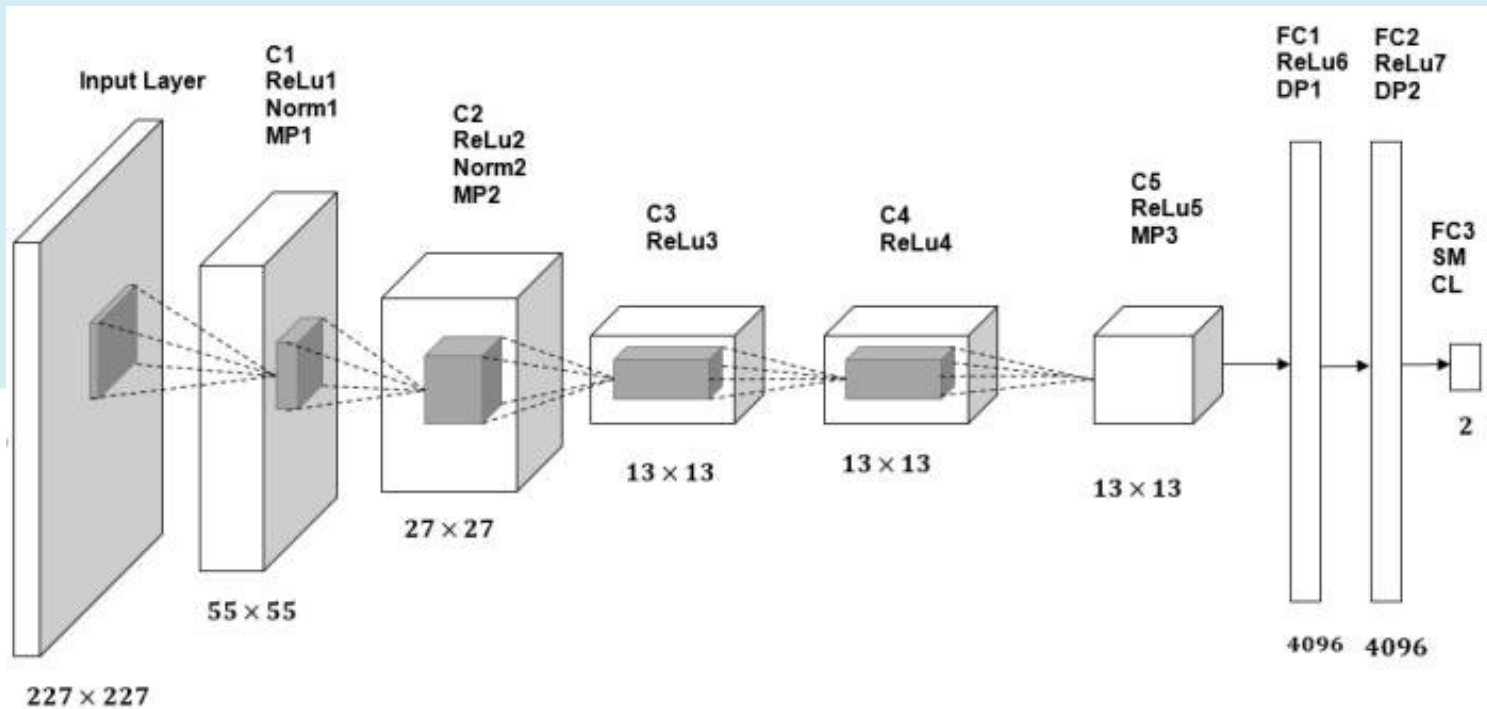
2 filtres 3×3 = même champ qu'un 5×5
mais 2× moins de paramètres + 2 non-linéarités

VGG-16 : 16 couches, 138 M paramètres

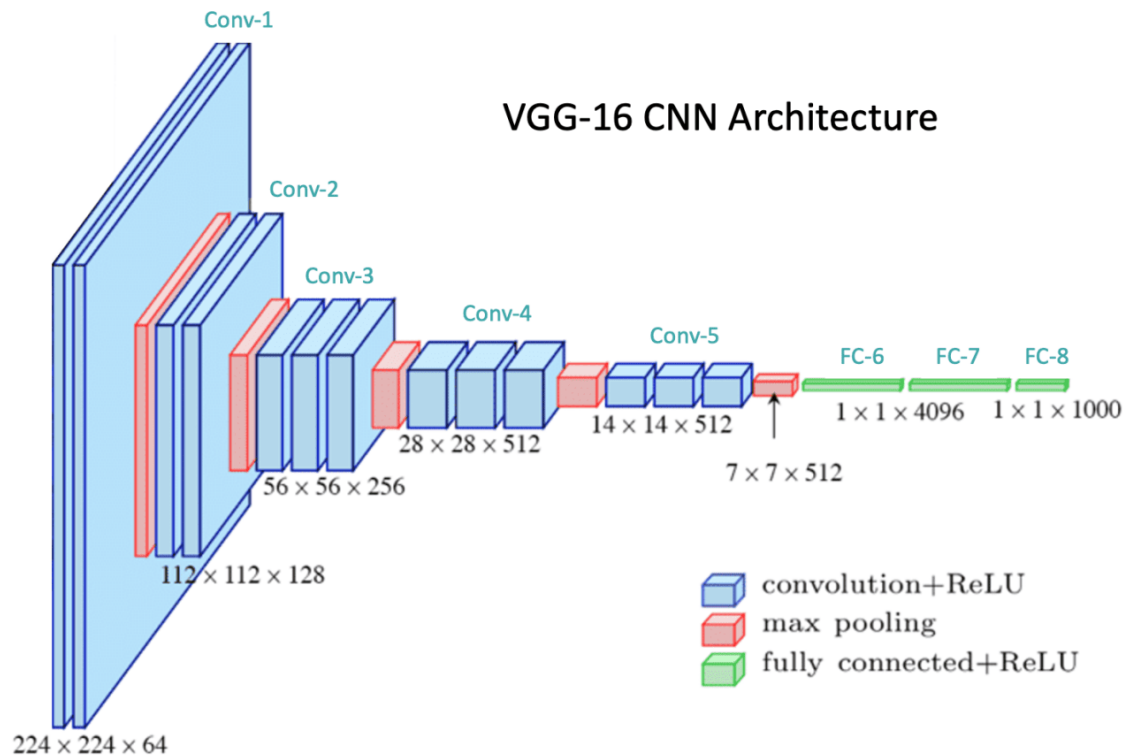
Encore utilisé comme backbone pour la segmentation de documents



LeNet-5 (1998)



AlexNet (2012)



VGGNet (2014)

ResNet et DenseNet — profondeur et réutilisation des features

ResNet (He et al., 2016) — l'article le plus cité en vision (> 100 000 citations)

Problème de dégradation : les réseaux très profonds (> 20 couches) ont une erreur d'entraînement PLUS élevée — ce n'est pas du surapprentissage, c'est une difficulté d'optimisation

Solution : le bloc résiduel

$$y = F(x, \{W_i\}) + x$$

Le réseau apprend la résiduelle $F(x) = y - x$ plutôt que $H(x)$ directement

Si la transformation optimale est l'identité : apprendre $F(x) \approx 0$ est trivial

Skip connections = raccourcis de gradient → rétropropagation sur 152 couches

ResNet-152 : 3.57% erreur top-5 — sous le niveau humain (~5%)

DenseNet (Huang et al., 2017)

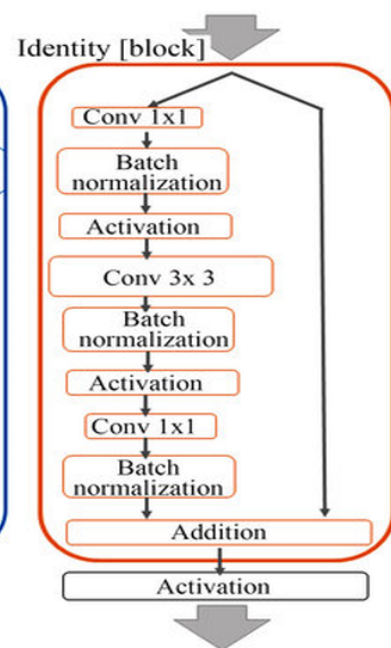
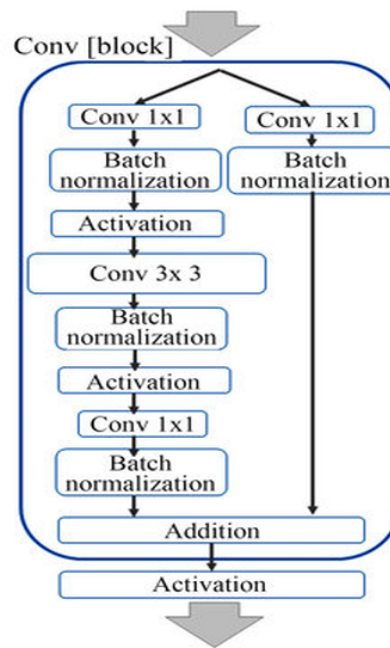
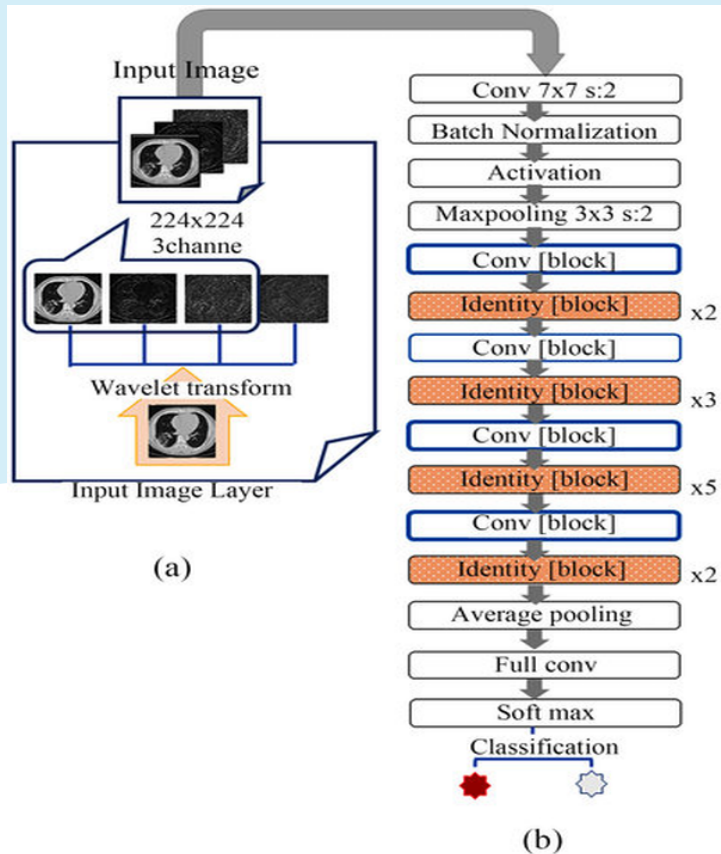
Dans un bloc dense, chaque couche reçoit les feature maps de TOUTES les couches précédentes (concaténation)

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

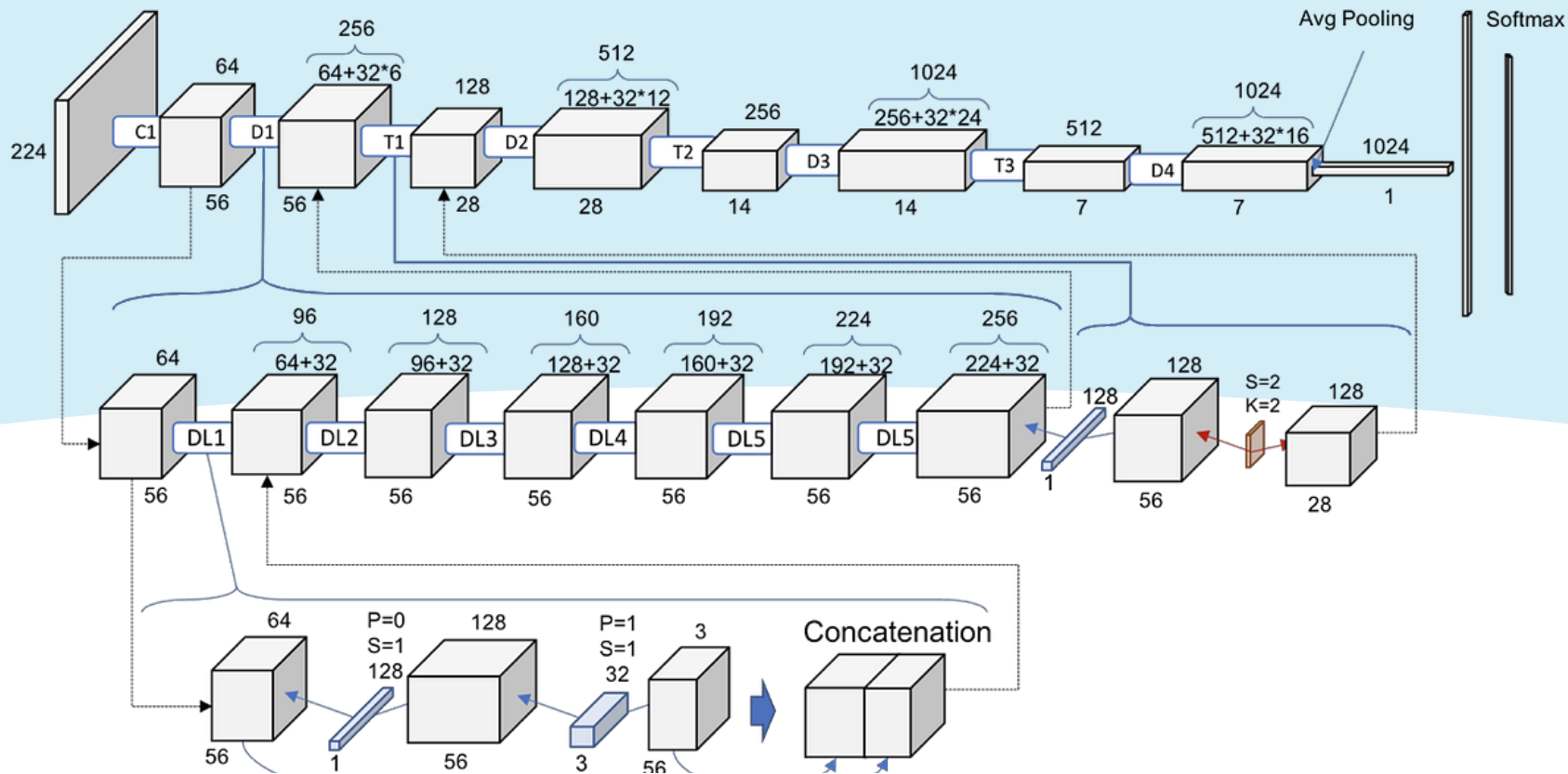
Avantages :

- Réutilisation maximale des features
- Gradients courts → entraînement facile
- Régularisation naturelle

Pertinent pour les documents : features bas niveau (bords, tracés) accessibles aux couches profondes



ResNet (2016)



DenseNet-121 (2017)

EfficientNet et architectures de segmentation

*Scaling composé pour la classification ·
Encodeur-décodeur pour la segmentation dense*

EfficientNet (Tan & Le, 2019)

Scaling composé : augmenter profondeur + largeur + résolution ensemble

profondeur : $d = \alpha^\psi$ · largeur : $w = \beta^\psi$ ·

résolution : $r = \gamma^\psi$

Contrainte : $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$ (budget computationnel fixe)

Backbone : MBConv avec Squeeze-and-Excitation

(attention par canal : pondère l'importance relative des canaux)

EfficientNet-B7 : 84.4% top-1 avec 8× moins de paramètres que ResNet comparable

U-Net (Ronneberger, 2015)

Architecture en U : encodeur + décodeur symétriques

Encodeur : conv + maxpool → ↓ résolution, ↑ sémantique

Décodeur : upsample + concat skip connection

Skip connections : transmettent les détails spatiaux fins préservés par l'encodeur

Référence pour la segmentation de layout dans les documents

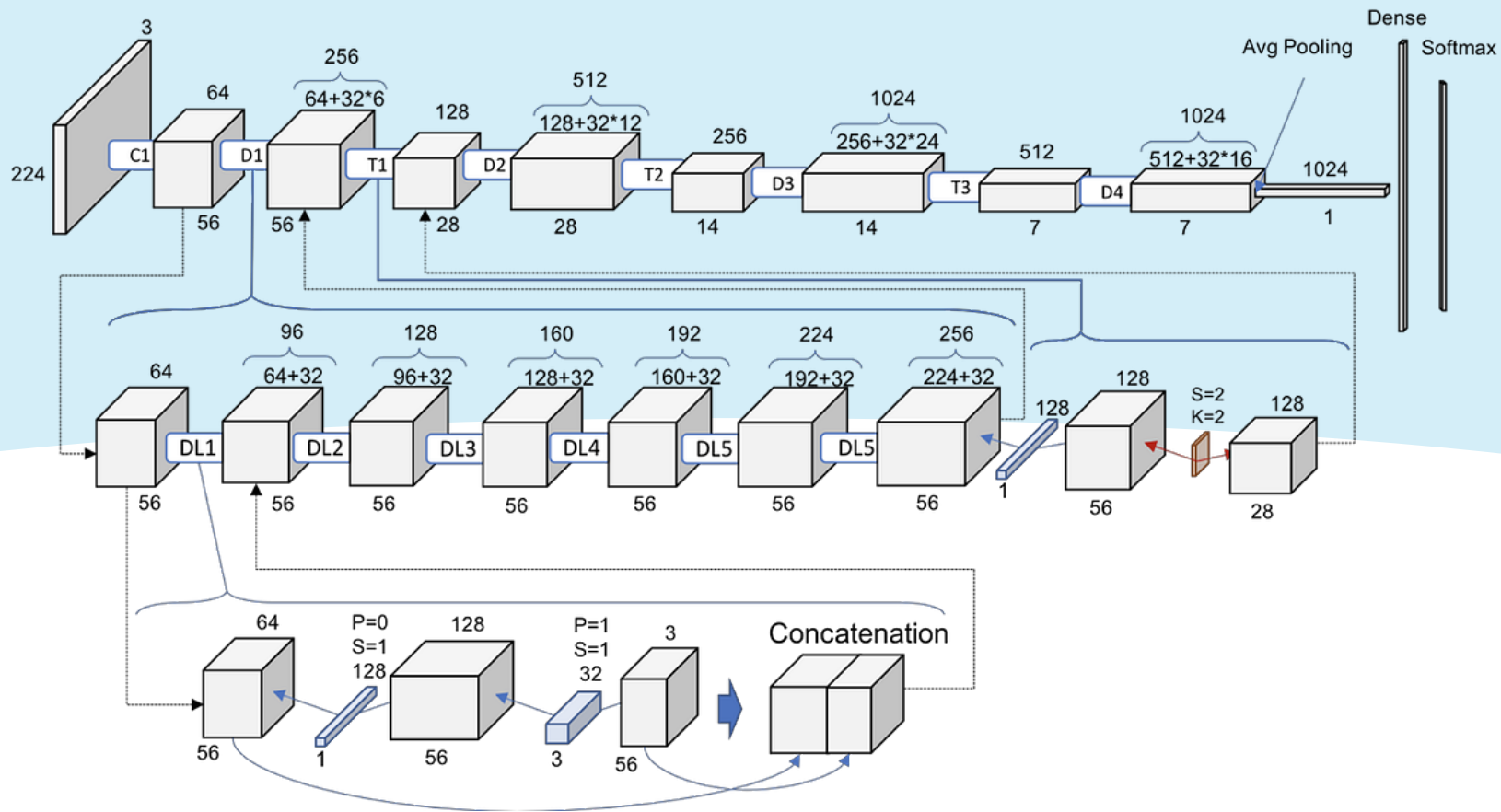
Variante ResUNet : encodeur ResNet-50 pré-entraîné

FCN — Fully Convolutional Network (Long, 2015)

Remplacement des couches FC par des convolutions 1×1
→ Sortie de même résolution que l'image

Upsampling ×32 pour revenir à la résolution native
FCN-16s / FCN-8s : skip connections intermédiaires

1ère segmentation sémantique dense entièrement convolutive
Précurseur de toutes les architectures de segmentation modernes



EfficientNet (2019)

3. Biais inductifs des CNN

Trois hypothèses structurelles — et leurs conséquences pour les manuscrits

Biais 1 — Invariance à la translation

La convolution est équivariante : $f(\text{Translate}(I)) = \text{Translate}(f(I))$ · Le pooling ajoute l'invariance

Quand c'est utile — images naturelles

Classification d'objets : un chat en haut à gauche = un chat en bas à droite

La position exacte de l'objet n'importe pas pour reconnaître sa classe

L'invariance permet la généralisation et réduit le besoin de données

Quand c'est problématique — manuscrits

La position relative des éléments est porteuse d'information

Dans un mot gothique : 1er jambage $\neq$ 3ème jambage $\rightarrow$ lettres différentes

Un modèle invariant à la translation ne « sait » pas où il se trouve dans la page

Conséquences concrètes pour les manuscrits médiévaux

Mise en page informative : zone en haut au centre = rubrique · zone en bas à droite = réclame $\rightarrow$ ignoré si invariance à la translation

Ordre de lecture (gauche $\rightarrow$ droite, haut $\rightarrow$ bas) : structure globale que l'invariance locale détruit

Lettrines : même forme graphique qu'une lettre normale, mais en position initiale de section $\rightarrow$ la position est le seul discriminant

Biais 2 & 3 — Localité et partage des paramètres

Un filtre $k \times k$ ne voit que k^2 pixels · Le même filtre est appliqué à toutes les positions spatiales

Biais 2 — Localité

Hypothèse : les features utiles sont locales — un pixel est plus corrélé à ses voisins qu'aux pixels distants

Utile : bords, textures, formes simples dans les images naturelles

Problème pour les manuscrits :

- Ligatures inter-mots : frontière de mot non visible localement
- Ambiguïté des minimes (iii/ui/ni/m) : contexte lexical à plusieurs centaines de px
- Abréviations : tilde au-dessus d'une lettre renvoie à une information absente
- Cohérence globale de ligne : hauteur et espacement sont des propriétés de ligne entière

Biais 3 — Partage des paramètres

Hypothèse : les features utiles sont les mêmes indépendamment de leur position

Le même filtre est appliqué à toutes les positions → statistiques supposées stationnaires

Utile : bords, textures uniformes dans les images naturelles

Problème pour les manuscrits :

- s long (ſ) : même forme graphique que s rond, mais n'apparaît qu'en début/milieu de mot — position-dépendant
- Lettrines : grande lettre initiale de section → interprétation dépend de la position dans la page
- Les statistiques spatiales des manuscrits sont non stationnaires

Coût du contexte global et forces des CNN sur les documents

Coût computationnel du contexte global

FLOPs conv $k \times k$: $2 \times H \times W \times C_{in} \times C_{out} \times k^2$ (quadratique en la résolution)

Pour capter la page entière : $O(\log N)$ couches de pooling OU $O(N/k)$ couches sans pooling

En pratique : travail sur 224×224 ou 448×448 — réduction $100 \times$ à $400 \times$ par rapport au scan original

Sliding window : ignore les dépendances entre fenêtres (mots coupés)

Ce que les CNN font bien sur les documents

Classification de type de document (charte, registre...) : texture globale, pas de localisation fine

Segmentation de lignes sur pages simples : U-Net CNN $\approx$ Transformer, moins coûteux

Extraction de features pour clustering : ResNet-50 sans fine-tuning
→ regroupement par main cohérent

Reconnaissance de caractères isolés (tâche auxiliaire de paléographie)

Conclusion : quand choisir un CNN pour les documents ?

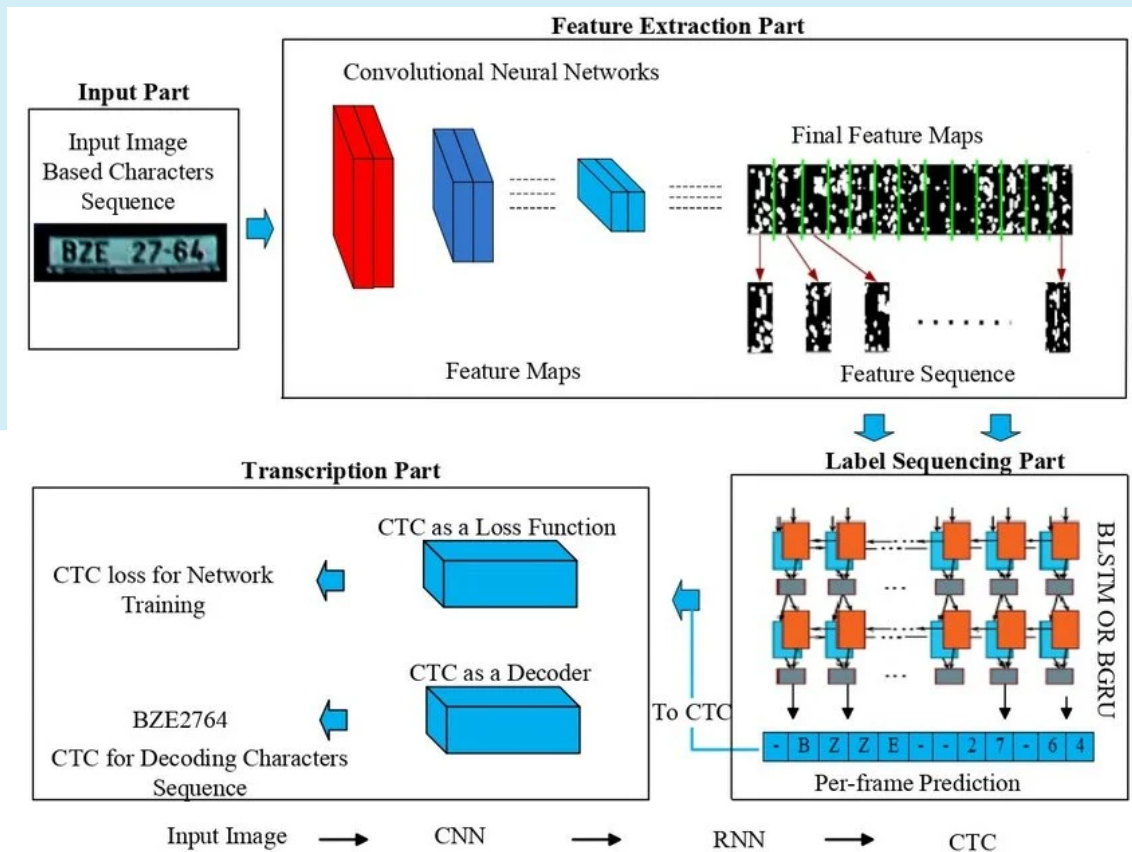
CNN adapté : tâches de classification globale, pas de localisation fine, peu de données, peu de ressources computationnelles

CNN insuffisant : HTR end-to-end, analyse de layout complexe, documents avec mise en page non uniforme, dépendances longue portée

DINOv2 / TrOCR (Transformers) prennent le relais pour les cas difficiles — voir section 2.2

4. Architecture hybride CNN-RNN

La référence pré-Transformer pour la reconnaissance de texte manuscrit



Convolutional Recurrent Neural Network (2015)

CRNN — Architecture et CTC Loss

Convolutional Recurrent Neural Network (Shi et al., 2015) — architecture dominante de l'HTR avant les Transformers

Architecture CRNN

1. CNN (VGG-like) : extraction de features spatiales
→ Feature maps ($H/32 \times W/4 \times 512$)
2. Collapse vertical : aplatir la dimension hauteur
→ Séquence de vecteurs ($W/4 \times 512$), un par colonne d'image
3. BiLSTM (2 couches) : contexte gauche ↔ droite
→ Capture des dépendances horizontales
4. Dense → Softmax → distribution sur l'alphabet par colonne
5. CTC Decoding → texte final

CTC Loss — Connectionist Temporal Classification (Graves, 2006)

Problème : aligner une séquence de labels (texte) avec une séquence de prédictions (colonnes d'image) sans annotation d'alignement

Solution : marginaliser sur TOUS les alignements possibles

$$L_{CTC} = -\ln P(y|x) = -\ln \sum_{\pi} P(\pi|x)$$

où $B(\pi) = y$ (décodage : supprimer blancs et répétitions)

Calculé efficacement par programmation dynamique (forward-backward)

Avantage : pas besoin de segmenter le texte en caractères — supervision au niveau de la ligne

Limites du CRNN pour les manuscrits médiévaux

Trois limites structurelles — précisément ce que l'attention des Transformers vient résoudre

Biais horizontal fort

Le collapse vertical suppose une ligne parfaitement horizontale

Sur les manuscrits sur parchemin :

- Lignes ondulées (support souple)
- Lignes légèrement inclinées
- Déformations locales du support

→ Dégradation significative des performances sur les manuscrits non normalisés

Mémoire limitée des LSTM

Les longues lignes (> 100 caractères) posent des problèmes aux LSTM

Les dépendances très longue portée sont mal capturées par les états cachés

Exemple : une abréviation en fin de ligne qui réfère à un mot vu au début de la même ligne

→ Erreurs croissantes avec la longueur de la ligne

Pas de mécanisme d'attention

Le CRNN agrège les features de toute la ligne avec le même poids implicite

Il n'y a pas de mécanisme permettant de « focaliser » sur une zone spécifique pour produire un caractère donné

→ L'attention du Transformer (section 2.2) résout exactement ce problème

→ TrOCR (Transformers for OCR) remplace le BiLSTM par du self-attention

Récapitulatif — Pourquoi les CNN atteignent leurs limites sur les manuscrits

Propriété CNN	Avantage général	Problème pour les manuscrits
Localité des filtres	Efficacité computationnelle	Contexte global nécessaire pour lever les ambiguïtés (minimes, abréviations)
Invariance à la translation	Robustesse aux décalages	Position porteuse d'information (ordre de lecture, lettrines, rubriques)
Partage des paramètres	Réduction du nb de paramètres	Statistiques spatiales non stationnaires (s long début ≠ fin de mot)
Réduction de résolution (pooling)	Montée en sémantique	Perte irréversible des détails fins des lettres
Champ réceptif limité	Extraction de features locales	Impossible de capturer la structure globale de la page
Pré-entraînement ImageNet	Bonne initialisation générale	Distribution shift fort : images naturelles ≠ scans de manuscrits